Database-Native Authorization for Human and Autonomous Principals: The SHRBAC Model

Ross Slaney

***Abstract*—Enterprise applications must answer two authorization questions: "can this principal act on this resource?" (point check) and "which resources can this principal access, filtered, sorted, and paginated?" (list filtering). The latter is harder: it requires composing authorization with application-defined filtering, sorting, and pagination into a single efficient query. We present SHRBAC (Scoped Hierarchical Role-Based Access Control), an authorization model whose structural constraints—tree-structured resources, flat roles, non-recursive groups, and scoped temporal grants—make enforcement a composable database predicate with bounded cost, and whose polymorphic principals extend the same grant semantics to human users, service accounts, and autonomous agents. We formalize the model, prove soundness and completeness of an inline Table-Valued Function (TVF) enforcement, and show that under cursor pagination, bounded principal sets and grant multiplicity, and indexed nested-loop plans, per-page cost is $O(k \cdot D)$. Empirical evaluation on SQL Server 2022 at 1.2M resource nodes (D=5) and 1.5M (D=10) confirms N-independence across three orders of magnitude in dense-access workloads.**

***Index Terms*—access control, RBAC, authorization, query composition, autonomous principals**

I. Introduction

A. The Problem

Modern B2B SaaS applications require authorization systems that answer two fundamentally different questions:

1) **Point check**: "Can principal p perform action a on resource r?"—a boolean decision.
2) **List filtering**: "What resources of type T can principal p access, given search criteria C, sorted by S, paginated to page k?"—a set-returning query integrated with application data.

The first is well-understood: Google's Zanzibar [1] and its descendants handle millions of point checks per second through graph traversal and caching. The second—the **list filtering problem**—is trivial for flat access models (`WHERE tenant_id = @currentTenant` composes directly) but becomes substantially harder when access derives from grants at varying hierarchy levels, resolved through group memberships, under temporal constraints.

Increasingly, principals are not only human users but **autonomous AI agents** that enumerate and filter large resource sets in loops. SHRBAC introduces no agent-specific policy semantics; it treats agents as first-class principals, so the same scoped, temporal grant model applies to human and autonomous callers, and the **principle of least privilege** [2] is enforced structurally: narrow grants (specific subtree, limited role, bounded duration) guard against broad, long-lived authority.

When authorization logic resides outside the database, an impedance mismatch arises. Industry bridges this gap through ID lookups, batch post-filtering, partial policy evaluation, or materialized permission views—each trading scalability, consistency, or complexity. SHRBAC instead chooses structural constraints so that per-page enforcement cost is $O(k \cdot D)$ under bounded local parameters—linear in page size and tree depth, independent of total resource count N and policy set size.

Notation. Throughout the paper, N denotes total entities or resource nodes under evaluation, k the requested page size, D the maximum resource-tree depth, $M = |resolve(p)|$ the number of effective principals for caller p, $G_{\max}$ the maximum active grant multiplicity for the same principal-resource pair, and σ the selectivity of authorized rows.

B. Contributions

1) **Constraint-driven model.** We introduce SHRBAC, whose four structural constraints—tree-structured resources, non-recursive principal groups, flat roles, and scoped temporal grants (principal $\times$ role $\times$ resource $\times$ time)—are deliberately chosen so that enforcement admits a fixed, schema-level relational artifact with bounded cost independent of policy set size. Polymorphic principals make users, groups, service accounts, and autonomous agents first-class grant recipients with least-privilege, time-bounded authority.
2) **Formalization with proved enforcement.** Access evaluation is two-dimensional—upward over a bounded-depth resource tree, outward over effective principals. We realize enforcement as a single inline Table-Valued Function `EXISTS` predicate defined at schema time and prove it sound and complete; grants are data rows, not predicate terms, requiring no runtime compilation, post-filtering, or external graph traversal.
3) **Complexity theorem for list filtering.** Per-row cost is bounded at $O(D \cdot M \cdot G_{\max})$; under cursor pagination, per-page cost is $O(k \cdot D)$ when M and $G_{\max}$ are bounded—independent of resource count N, policy set size, and page depth. This formal bound for the list-filtering problem is absent in prior RBAC, ABAC, and ReBAC models.
4) **Production-scale empirical validation.** At 1.2M resources (D=5) and 1.5M resources (D=10), we confirm N-independence across three orders of magnitude, linear scaling in k and D, constant per-CTE-hop cost, cursor-depth independence, and grant-breadth independence in tested dense-access workloads.

II. Background and Related Work

A. RBAC Foundations

Sandhu et al. [5] defined the RBAC96 family: RBAC0 (core), RBAC1 (hierarchical roles), RBAC2 (constraints), and RBAC3 (combined). The NIST standard [6] formalized Core and Hierarchical RBAC. **Critical gap:** RBAC96 defines only a role hierarchy; the resource space is flat with no concept of resource hierarchy or scope.

B. Hierarchical Models

ROBAC (Zhang et al. [3]) introduced the (user, role, organization) triple with grants cascading down an organization hierarchy. SHRBAC's (principal, role, resource) grant is structurally identical, but ROBAC defines a policy semantics without constraining the model to admit a fixed enforcement artifact with bounded evaluation cost. SHRBAC's novelty is the alignment of model constraints with relational query-planning guarantees, extended to polymorphic principals. **RRBAC** (Solanki et al. [7]) formalized resource hierarchies with grant cascade but did not address polymorphic principals, query composition, or enforcement mechanisms. **OrBAC** (Kalam et al. [8]) generalized RBAC with five hierarchies; its expressiveness exceeds what TVF-based enforcement requires—SHRBAC is a practical engineering subset.

C. Query Modification and Enforcement

Stonebraker and Wong [4] introduced query modification for INGRES—the ancestor of Oracle VPD and SQL Server/PostgreSQL row-level security. Rizvi et al. [9] distinguished the *Truman model* (silent filtering, which SHRBAC implements) from the *Non-Truman model* (query rejection). Pappachan et al. [10] showed that naive predicate injection does not scale with thousands of policies; SHRBAC's predicate is a single TVF invocation regardless of policy count.

D. Zanzibar, ReBAC, and List Filtering

Google's Zanzibar [1] uses relationship tuples with a userset rewrite system; Fong [11] formalized ReBAC using modal logic. Zanzibar's `tuple_to_userset` handles resource hierarchies through general graph traversal. SHRBAC constrains resources to a tree and uses typed roles rather than arbitrary relation composition—tree traversal has bounded depth, which is what makes TVF enforcement tractable. Composing authorization into list queries has no canonical academic name: vendors call it "ACL-aware filtering," "list filtering," or a "query plan" problem [12]. At scale, SpiceDB recommends progressively more complex strategies (LookupResources → CheckBulkPermissions → Materialize). SHRBAC is designed so that the integration surface is a relational predicate—composable by construction.

E. Summary of Gaps

Table I summarizes the capability comparison. To our knowledge, no prior model jointly formalizes resource hierarchy with grant cascade, polymorphic principals (humans, groups, and agents as first-class grant recipients), and a concrete query-composable enforcement mechanism with a complexity bound.

III. The SHRBAC Model

A. Basic Sets

The model comprises: a finite set $\mathcal{P}$ of *principals*, each typed $\tau(p) \in \{\text{user}, \text{group}, \text{service\_account}, \text{agent}\}$, where an agent is an autonomous process (AI agent, orchestration pipeline, or background service) participating in the same grant relation as human users under the same scoping and temporal constraints; a finite set $\mathcal{R}$ of *roles*; a finite set $PERM$ of atomic *permissions* (e.g., PROJECT_VIEW); a finite set RES of *resources* forming nodes in a rooted tree, each carrying a type from RT (e.g., `portal_root`, `agency`, `project`); and the time domain $\mathcal{T}$ (UTC timestamps).

B. Resource Hierarchy

The resource hierarchy is a rooted tree $(RES, parent)$ with $parent : RES \to RES \cup \{\bot\}$ and $parent(r_0) = \bot$ for the root r_0. Define $ancestors(r) = \{parent^i(r) \mid i \geq 0 \wedge parent^i(r) \neq \bot\}$ (note $r \in ancestors(r)$), $descendants(r) = \{r' \in RES \mid r \in ancestors(r')\}$, and $depth(r) = |ancestors(r)| - 1$. The tree is bounded: $depth(r) \leq D$ for all r; in practice $D = 3$–5 for typical SaaS and up to 10–15 for deep enterprise hierarchies. An **authorized entity** is a row in an application table carrying a `ResourceId` referencing a node in the resource tree.

C. Role-Permission Assignment

$PA \subseteq \mathcal{R} \times PERM$ is the role-permission assignment. Each permission has an associated resource type: $applicable : PERM \to RT$. Roles are **flat**—no role hierarchy. The resource hierarchy provides the inheritance dimension, avoiding the combinatorial complexity of dual hierarchies.

D. Principal Resolution

Groups are themselves principals. $members(g) = \{u \in \mathcal{P} \mid \tau(u) = \text{user} \wedge u \text{ is a member of } g\}$. Membership is not recursive—groups cannot contain other groups.

Principal resolution expands a user to their effective principal identities:

$$resolve(p) = \{p\} \cup \{g \in \mathcal{P} \mid \tau(g) = \text{group} \wedge p \in members(g)\}$$

For non-user principals (groups, service accounts, agents), $resolve(p) = \{p\}$.

E. Grants

$G \subseteq \mathcal{P} \times \mathcal{R} \times RES \times (\mathcal{T} \cup \{\bot\}) \times (\mathcal{T} \cup \{\bot\})$

A grant $g = (p, role, res, t_{from}, t_{to})$ assigns role $role$ to principal p at resource res, effective during the closed (inclusive) interval $[t_{from}, t_{to}]$; a $\bot$ bound means unbounded.

Active grants at time t:

$$active(t) = \{(p, role, res) \mid (p, role, res, t_f, t_t) \in G \\ \wedge\ (t_f = \bot \vee t_f \leq t) \wedge (t_t = \bot \vee t_t \geq t)\}$$

TABLE I
CAPABILITY COMPARISON ACROSS AUTHORIZATION MODELS. "✓" INDICATES NATIVE SUPPORT.

Capability	RBAC96	ROBAC	RRBAC	OrBAC	ReBAC	SHRBAC
Role hierarchy	RBAC1+	✓	✓	✓	N/A	No (flat)
Resource hierarchy	No	Org hier.	✓	Org hier.	Graph	✓ (tree)
Grant cascade	No	✓	✓	✓	`tuple_to_userset`	✓
Polymorphic principals	No	No	No	Partial	Usersets	✓
Group-as-principal	No	No	No	No	Via tuples	✓
Agent-as-principal	No	No	No	No	No	✓
Temporal grants	No	No	No	Contexts	No	✓
Query composition	No	No	No	No	No	✓
Formal enforcement	No	No	No	No	Graph trav.	Query mod.

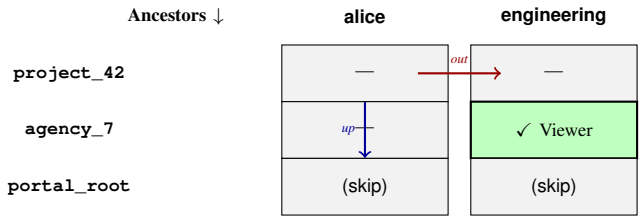

Fig. 1. Two-dimensional resolution for the running example: principals expand outward (columns), ancestors are walked upward (rows), and the first match short-circuits.

The grant triple (principal, role, resource) enforces least privilege: authority is scoped to a specific subtree (not global), a specific role (not all permissions), and optionally a specific time window (not permanent).

F. Access Evaluation Function

Definition 1 (Access Decision)**.** *Given principal p, permission $perm$, resource r, and time t:*

$$\begin{aligned} allowed(p, perm, r, t) = \exists\, p' \in resolve(p), \\ \exists\, r' \in ancestors(r),\ \exists\, role \in \mathcal{R} : \\ (p', role, r') \in active(t) \wedge perm \in perms(role) \end{aligned}$$

Access is granted if *any* effective identity has *any* active grant at *any* ancestor of the target resource with a role including the requested permission. This is **two-dimensional resolution**: simultaneously walking UP the resource tree and expanding OUT the principal set.

Running example. Let `portal_root` → `agency_7` → `project_42`, with Alice in group `engineering` and a single grant (`engineering`, VIEWER, `agency_7`). Evaluating $allowed(\text{alice}, \text{PROJECT\_VIEW}, \texttt{project\_42}, now)$ searches the 2×3 grid $resolve(\text{alice}) \times ancestors(\texttt{project\_42})$ and finds the VIEWER grant at (`engineering`, `agency_7`): access is granted—the grant cascades to the descendant `project_42` (Fig. 1).

G. Properties

Property 1 (Monotonicity of Hierarchy)**.** *If $allowed(p, perm, r, t)$ and $r' \in descendants(r)$, then the same grant provides access at r' (immediate from transitivity of the ancestor relation).*

Property 2 (Monotonicity of Groups)**.** *Adding p to group g preserves existing access and may grant additional access (existential quantification over an expanded $resolve(p)$).*

Property 3 (Bounded Evaluation)**.** *For any access decision, evaluation examines at most $|resolve(p)| \times |ancestors(r)| \times G_{\max}$ grant entries, giving $O(M \cdot D \cdot G_{\max})$.*

IV. QUERY-COMPOSABLE ENFORCEMENT VIA TVF

A. The Table-Valued Function

The enforcement mechanism realizes Definition 1 directly as an inline Table-Valued Function (iTVF), evaluated per row as an `EXISTS` predicate:

```
CREATE FUNCTION dbo.fn_IsResourceAccessible(
  @ResourceId NVARCHAR(128), @PrincipalIds NVARCHAR(MAX),
  @PermissionId NVARCHAR(128), @Now DATETIME2)
RETURNS TABLE AS RETURN (
  WITH ancestors AS (
    SELECT Id, ParentId, 0 AS Depth
      FROM Resources WHERE Id = @ResourceId
    UNION ALL
    SELECT r.Id, r.ParentId, a.Depth + 1
      FROM Resources r
      INNER JOIN ancestors a ON r.Id = a.ParentId
      WHERE a.Depth < 10)
  SELECT TOP 1 a.Id FROM ancestors a
    INNER JOIN Grants g ON a.Id = g.ResourceId
    INNER JOIN RolePermissions rp ON g.RoleId = rp.RoleId
  WHERE g.PrincipalId IN (SELECT LTRIM(RTRIM(value))
          FROM STRING_SPLIT(@PrincipalIds, ','))
    AND rp.PermissionId = @PermissionId
    AND (g.EffectiveFrom IS NULL OR g.EffectiveFrom <=
         @Now)
    AND (g.EffectiveTo IS NULL OR g.EffectiveTo >= @Now))
```

The recursive CTE walks UP from the target resource to the root (at most D levels), joining against active grants at each ancestor. `TOP 1` provides existential semantics—one matching grant suffices. The caller-controlled `@Now` parameter ensures deterministic evaluation across all rows in a query. As an inline TVF, the optimizer composes its body into the calling query's execution plan: the application layer resolves group memberships once per request, then appends the TVF as an `EXISTS` predicate alongside application-defined filters, cursor pagination, and sorting—authorization and application logic execute as a single database operation, one round-trip, zero external calls. We use SQL Server as an exemplar because its optimizer aggressively inlines iTVFs; the enforcement requires only recursive CTEs and predicate inlining, both supported by PostgreSQL and other modern engines.

B. Correctness

Theorem 1 (Soundness and Completeness)**.** *For entities with $depth \leq 10$, the TVF returns a non-empty result for entity e iff $allowed(p, perm, e.ResourceId, now) =$ true.*

Proof. The recursive CTE (guard `Depth < 10`) produces exactly $ancestors(e.ResourceId)$; the joins against `Grants` and `RolePermissions` with the temporal predicates implement the existential quantification of Definition 1 over $active(t)$. A non-empty result therefore witnesses a satisfying $(p', role, r')$ triple, and conversely any satisfying witness matches the corresponding CTE row, principal filter, and temporal and role-permission predicates by construction. □

V. Complexity Analysis

A. Per-Row TVF Cost

For a single entity with resource at depth $d \leq D$, the CTE performs $O(D)$ index seeks on `Resources(Id)`; joining the D ancestors against `Grants` and `RolePermissions` examines at most $D \cdot M \cdot G_{\max}$ candidates; and `TOP 1` short-circuits on the first match. **Per-row cost:** $O(D \cdot M \cdot G_{\max})$—with $D = 5$, $M = 10$, $G_{\max} = 3$, $\sim$150 index lookups.

B. Per-Page Cost Under Pagination

Theorem 2 (Per-Page Complexity)**.** *Assume (1) index coverage on Resources, Grants, RolePermissions, and entity ResourceId; (2) cursor pagination on an indexed ordering key; (3) nested-loops plans with index seeks. Then dense access ($\sigma \approx 1$) costs $O(k \cdot D \cdot M \cdot G_{\max})$ per page—$O(k \cdot D)$ when M and $G_{\max}$ are bounded constants—and sparse access costs $O(k/\sigma \cdot D \cdot M \cdot G_{\max})$, degenerating to $O(N \cdot D \cdot M \cdot G_{\max})$ when no rows are authorized.*

Proof. With cursor pagination the engine seeks to the cursor in $O(\log N)$ and evaluates candidates sequentially, each costing $O(D \cdot M \cdot G_{\max})$ by the per-row bound above; collecting k authorized rows at selectivity σ examines $\sim k/\sigma$ candidates, giving $O(k/\sigma \cdot D \cdot M \cdot G_{\max})$. The cursor start is resolved by index seek, not by scanning prior pages, so cost is independent of page depth and N. □

The per-page cost is *parameterized and predictable*—independent of N, linear in D. Alternatives instead pay an external-service round trip per page (LookupResources, batch post-filtering), a policy-compilation step (partial evaluation), or eventual-consistency denormalization infrastructure (materialized views); SHRBAC's cost is database-local with strong consistency.

VI. Empirical Evaluation

We evaluate SHRBAC on SQL Server 2022 (Docker, 4 vCPU, 8 GB RAM) across three tiers: (1) small-scale isolation tests (1K–100K entities) that individually vary each factor, (2) a production-scale workload at D=5 with 1.2M resource nodes, and (3) a deep-hierarchy workload at D=10 with 1.5M resource nodes. The environment is a controlled validation setup, not a production latency claim; Section VII discusses generalizability.

A. Benchmark Methodology

Measurement protocol. Each query: 3 warmup runs (discarded), 20 measured runs, each on a fresh connection, recording wall-clock elapsed time. We report median and P95. Page size $k = 20$ unless stated otherwise.

Query pattern. All list-filtering benchmarks execute the canonical authorized-list query: select the top k `Products` rows satisfying `EXISTS(fn_IsResourceAccessible(...))` with a cursor predicate (`Id > @cursor`) and `ORDER BY Id`. Point checks use the same TVF against a single ResourceId. Query plans were verified to use nested loops with index seeks.

Principal parameter passing. All reported benchmarks, including the M=11 and M=21 experiments, use the `STRING_SPLIT` TVF of Section IV (no table-valued parameters), so M-scaling conclusions are limited to the tested range. `STRING_SPLIT` can produce poor cardinality estimates; table-valued parameters are recommended for larger principal sets.

Hierarchies. D=5: root → 15 chains → 150 regions → 15K stores → 1.2M products = **1,215,166 resources**. D=10: root → 5 divisions → 25 regions → 125 districts → 500 areas → 2K zones → 12K stores → 60K departments → 240K sections → 1.2M products = **1,514,656 resources**. Resources and domain rows are bulk-inserted; `UPDATE STATISTICS` is run before benchmarks.

Isolation tests (Benchmarks 1–7) use the same retail domain model scaled to the target depth (e.g., D=3: root → Regions → Stores → Products(N)). Each level is a real domain table whose rows map to unique resources, and the benchmark query always targets Products, so small-scale tests exercise the same schema and query patterns as the production-scale ones.

B. Isolation Tests (1K–100K)

Controlled single-factor experiments, re-seeded per configuration, confirm the core predictions: N-independence across 1K–100K (3.32–3.57ms median at D=3; Fig. 2), monotonic depth scaling from 2.45ms (D=1) through 3.17ms (D=3) to 4.21ms (D=5) consistent with $O(k \cdot D)$, and a negligible principal-set effect (3.32–3.47ms across M=1–21). P95 stayed below 6.2ms in every configuration.

C. Production-Scale at D=5 (1.2M Resources)

Principals range from company administrators (all 1.2M products accessible) to store managers ($\sim$80 products); page-size results appear in Table II. **Scope level**: despite a $15{,}000\times$ difference in accessible product count, latency varies minimally—company admin 3.47ms, chain manager 3.20ms, region manager 3.11ms, store manager 2.02ms median (P95 $\leq$ 6.34ms)—store-level grants fastest because the CTE short-circuits. **Cursor depth** is flat: 3.08ms at page 1, 3.25ms at page 50, 3.48ms at page 500.

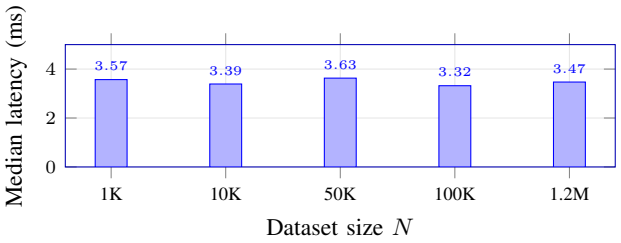

Fig. 2. Per-page median latency (k=20, cursor) is flat at ∼3.3–3.6ms across three orders of magnitude; the small increase at 1.2M is a depth effect (D=5 vs D=3), not resource count.

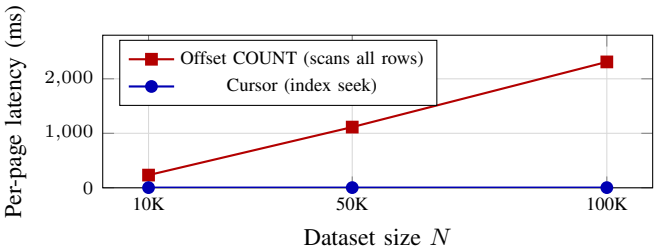

Fig. 3. Offset COUNT grows linearly: 231ms (10K) → 1,114ms (50K) → 2,310ms (100K); cursor holds at ∼3.3ms (700× faster at 100K).

TABLE II
PAGE SIZE COMPARISON: D=5 VS. D=10.

k	D=5 (ms)	D=10 (ms)	Ratio
10	2.28	3.44	1.51×
20	3.47	5.69	1.64×
50	6.89	11.80	1.71×
100	13.71	22.01	1.61×

D. Deep Hierarchy at D=10 (1.5M Resources)

Table II compares identical queries across both tree depths.

Page size scales linearly at both depths: k=10 → 100 yields 6.01× at D=5 and 6.40× at D=10, closely tracking $O(k)$. The D=10/D=5 ratio (1.51–1.71×) is below the theoretical $9/4 = 2.25\times$ because fixed per-page overhead is proportionally larger at D=5; the gap narrows at larger k. Per-row cost is ∼0.14ms at D=5 (∼0.034ms per CTE hop) and ∼0.22ms at D=10 (∼0.024ms per hop), giving $O(k \cdot D)$ a concrete, measurable constant. Scope-level and cursor-depth results at D=10 mirror D=5: scope varies minimally (3.06–5.54ms), and cursor depth is flat (5.11–5.38ms across pages 1–500).

E. Point Access Checks

The TVF also serves as a point check. Across target depths 0–4 at D=5 (1.2M resources), point checks complete in 0.86–1.31ms median; across depths 0–9 at D=10 (1.5M resources), in 0.89–1.46ms—all **under 1.5ms median** even at depth 9. Grant set size (1–20 active grants; 1.19–1.38ms) and principal set size (M=1–21; 0.98–1.28ms) are flat in the tested `STRING_SPLIT` range. The CTE examines only the target resource's ancestor chain ($\leq D$ nodes), not the grant table at large.

F. Analysis

N-independence extends to 1.5M resources (Fig. 2). With 1,215,166 unique resource nodes at D=5, list filtering median is 3.47ms (k=20)—compared to 3.39ms at N=10K with D=3 in isolation tests. The small increase is attributable to depth, not resource count. At D=10 with 1,514,656 resources, per-page cost is 5.69ms—again a depth effect.

Grant breadth is not local grant multiplicity. The breadth experiments vary the number of distinct resource scopes granted to a principal: at D=5, 1 vs. 10 chain scopes changes median latency by 0.09ms (3.31 vs. 3.40ms); at D=10, 1 vs. 5 division scopes changes it from 5.00ms to 5.76ms. Unrelated grant rows at other scopes are not scanned when evaluating a candidate resource. $G_{\max}$—multiple active grants for the same principal-resource pair—is a different factor: point checks showed no sensitivity for 1–20 active grants, though the evaluation does not refute the formal $O(G_{\max})$ factor.

Narrow grants can improve constants. A chain-level grant (3.12ms) is 37% slower than a single store grant (2.28ms): the CTE walks UP from the product, so a store-level grant matches at the first ancestor hop while a chain-level grant requires 3–4 hops. The 100-store case (4.27ms vs. 2.50ms for 10 stores) shows many disjoint narrow grants are not monotonically faster; selectivity and plan estimates affect constants without violating the per-row bound.

Principal set size is negligible in the tested range. M=1 vs M=11 produces 3.36 vs. 3.44ms (D=5) and 5.63 vs. 5.85ms (D=10). Because these experiments used `STRING_SPLIT`, this is an empirical observation for small effective-principal sets ($M \leq 21$), not a claim that parameter passing is irrelevant for large M.

Cursor vs. offset pagination (Fig. 3). Offset COUNT costs grow linearly with N while cursor pagination remains at ∼3.3ms: 700× faster at N=100K.

TVF vs. materialized. The materialized approach is ∼3.4× faster per-page (1.01ms vs. 3.42ms at N=100K) but requires denormalization infrastructure. Both are N-independent under cursor pagination; the TVF provides strong consistency with zero infrastructure overhead.

VII. DISCUSSION

A. Constraints as Architectural Choice

SHRBAC's performance guarantees are a direct consequence of four structural constraints, each eliminating a source of unbounded computation in the enforcement path:

Theorem 3 (Constraint-Composability Tradeoff)**.** *SHRBAC intentionally disallows (C1) DAG-structured resources, (C2) recursive group membership, (C3) role hierarchies, and (C4) arbitrary attribute predicates. In return, (G1) CTE traversal is bounded at $O(D)$; (G2) principal resolution is a single join producing a fixed set; (G3) the TVF body is fixed at schema design time—no runtime policy compilation; and (G4) enforcement is a standard SQL `EXISTS` subquery*

composable with arbitrary `WHERE`*,* `ORDER BY`*, and pagination.*

Relaxing any constraint breaks its guarantee: DAGs yield unbounded ancestor paths (G1), nested groups recursive resolution (G2), role inheritance transitive closure at evaluation time (G3), arbitrary predicates runtime policy compilation (G4). These constraints are not artificial—they codify the *de facto* structure of most multi-tenant B2B SaaS systems: containment trees rooted at a tenant boundary, flat membership groups, and roles that enumerate permissions. SHRBAC formalizes this common but undocumented pattern.

B. Relationship to ABAC and ReBAC

SHRBAC evaluates two attributes—role and resource-scope—making it a two-attribute constrained ABAC system [13]. The resource tree can be viewed as a constrained ReBAC graph in which every relationship is `parent_of` typed and the graph is a tree; that constraint (bounded depth, deterministic ancestor chains) is what makes TVF enforcement tractable. For arbitrary relationship graphs, ReBAC is more appropriate; for organizational hierarchies, SHRBAC's constraint matches the domain.

C. SHRBAC for Autonomous Principals

An agent is a principal with $\tau(p) =$ agent in the same grant relation as human users. SHRBAC therefore supports database authorization for agents that query or mutate application resources; it does not address prompt injection, tool-selection safety, model alignment, delegation-chain reasoning, or semantic policy generation. Within this scope, three properties follow directly. *Least-privilege delegation*: an agent receives only the role it needs, at only the subtree it operates on, for only the duration of its task (e.g., a 15-minute grant at one project subtree). *Predictable query cost*: under the same pagination and indexing assumptions as human callers, the per-page bound makes agent-driven enumeration loops independent of total resource count. *Auditability*: every agent action traces to a specific (agent, role, resource) grant, and when $\mathit{EffectiveTo} < now$, access ceases immediately without token-revocation infrastructure.

D. Scope and Limitations

The constraints exclude expressiveness by design: DAG resources such as matrix management are not supported (C1; ReBAC is more appropriate there), deeply nested groups require extending $resolve(p)$ with CTE traversal (C2), roles must enumerate permissions rather than inherit (C3), and dynamic attribute conditions such as IP or device type are not expressible (C4). For very deep hierarchies, a closure table replaces the $O(D)$ recursive CTE expansion with $O(1)$ lookup. The evaluation focuses on dense authorized-list workloads: Theorem 2 predicts $O(k/\sigma \cdot D \cdot M \cdot G_{\max})$ for sparse access and $O(N \cdot D \cdot M \cdot G_{\max})$ when no rows are authorized—deny-heavy workloads remain an empirical failure mode to stress-test. Finally, all measurements use SQL Server 2022 in Docker (4 vCPU, 8 GB RAM); the measured constants are not production SLAs, and although PostgreSQL and MySQL 8.0+ support the required primitives (recursive CTEs and predicate inlining), cross-DBMS validation is future work.

VIII. Conclusion

We presented SHRBAC, a formal authorization model at the intersection of hierarchical RBAC, polymorphic principal resolution, and Stonebraker's query modification, whose structural constraints allow per-page enforcement cost to simplify to $O(k \cdot D)$ under bounded principal sets, bounded local grant multiplicity, cursor pagination, correct indexing, and stable nested-loop plans. Empirical evaluation at 1.2M (D=5) and 1.5M (D=10) resources confirms the predicted behavior for dense-access workloads: per-page latency is N-independent across three orders of magnitude, scales linearly with k and D, and is unaffected by cursor depth or grant breadth in tested configurations.

As autonomous agents become principals in enterprise systems, the need for scoped, auditable, time-bounded authorization with efficient list filtering will intensify. SHRBAC provides this database-native substrate by applying the same scoped grant model to autonomous and human principals.

AI Disclosure

In accordance with IEEE policy, the author discloses that AI tools (Claude, Anthropic) assisted with manuscript preparation and editing; all technical content, formal definitions, proofs, implementation, and experimental design are the author's work.

References

[1] R. Pang et al., "Zanzibar: Google's Consistent, Global Authorization System," in *Proc. USENIX ATC*, pp. 33–46, 2019.

[2] J. H. Saltzer and M. D. Schroeder, "The Protection of Information in Computer Systems," *Proc. IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.

[3] Z. Zhang et al., "ROBAC: Scalable Role and Organization Based Access Control Models," in *Proc. CollaborateCom*, 2006.

[4] M. Stonebraker and E. Wong, "Access Control in a Relational Data Base Management System by Query Modification," in *Proc. ACM National Conf.*, 1974, pp. 180–187.

[5] R. Sandhu et al., "Role-Based Access Control Models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[6] D. F. Ferraiolo et al., "Proposed NIST Standard for Role-Based Access Control," *ACM TISSEC*, vol. 4, no. 3, pp. 224–274, 2001.

[7] N. Solanki et al., "Resource and Role Hierarchy Based Access Control for Resourceful Systems," in *Proc. IEEE COMPSAC*, pp. 396–401, 2018.

[8] A. Abou El Kalam et al., "Organization Based Access Control," in *Proc. IEEE POLICY*, 2003.

[9] S. Rizvi et al., "Extending Query Rewriting Techniques for Fine-Grained Access Control," in *Proc. ACM SIGMOD*, pp. 551–562, 2004.

[10] P. Pappachan et al., "Sieve: A Middleware Approach to Scalable Access Control for Database Management Systems," *PVLDB*, vol. 13, no. 12, 2020.

[11] P. W. L. Fong, "Relationship-Based Access Control: Protection Model and Policy Language," in *Proc. CODASPY*, pp. 191–202, 2011.

[12] AuthZed, "Protecting a List Endpoint," SpiceDB Documentation; Oso, "List Filtering," Oso Documentation; Cerbos, "Filtering Data Using Authorization Logic," Cerbos Blog. [Online].

[13] V. C. Hu et al., "Guide to Attribute Based Access Control (ABAC) Definition and Considerations," *NIST SP 800-162*, 2014.